

# 01\_Module\_1\_Intro\_to\_SRE

## Module 1 — Introduction to Site Reliability Engineering

Duration: 6 Hours

---

### 1.1 What is SRE?

Site Reliability Engineering is a discipline that applies software engineering principles to operations and infrastructure problems.

The canonical definition:

“SRE is what happens when you ask a software engineer to design an operations function.” — Ben Treynor Sloss, Google

#### 1.1.1 Origins

2003, Google. Their production systems had grown beyond what traditional operations could manage. The ops team was drowning in alerts, manual pages, and toil. Instead of hiring more operators, Google hired software engineers and told them: *run production like it’s a software engineering problem.*

The result was a new discipline that: - **Quantifies reliability** numerically (SLIs, SLOs, error budgets) - **Automates operations** instead of hiring for it - **Eliminates toil** as a first-class engineering priority - **Treats incidents** as learning opportunities, not failures

#### 1.1.2 SRE vs DevOps vs Platform Engineering

| Dimension             | DevOps                                | SRE                                       | Platform Engineering                      |
|-----------------------|---------------------------------------|-------------------------------------------|-------------------------------------------|
| Origin                | 2008, Agile ops                       | 2003, Google                              | 2020+, internal developer platforms       |
| Core goal             | Collaboration between dev & ops       | Quantified reliability at scale           | Developer velocity via self-service       |
| Key metric            | DORA metrics (deploy freq, lead time) | Error budget remaining                    | Developer satisfaction, platform adoption |
| Failure response      | Fix it fast, learn                    | Blameless post-mortem + error budget burn | Fix the platform, not the app             |
| Automation philosophy | Automate everything                   | Automate toil, keep engineering work      | Automate infrastructure, expose APIs      |

| Dimension   | DevOps     | SRE                                        | Platform Engineering                   |
|-------------|------------|--------------------------------------------|----------------------------------------|
| Scale focus | Team-level | Organisation-wide, mathematically rigorous | Organisation-wide, abstraction-focused |

**SRE is the implementation of DevOps with engineering rigour.** DevOps says “collaborate.” SRE says “here’s exactly how.”

---

## 1.2 The Three Core Principles

### 1.2.1 Embrace Risk

100% reliability is not a goal — it is a trap. The pursuit of absolute uptime:

- Freezes all change (any change risks downtime)
- Costs exponentially more for each additional nine (engineering cost ~10x per nine)
- Provides diminishing returns users cannot perceive (nobody feels the difference between 99.9% and 99.99%)

**The reliability/cost curve:**

Cost per nine of reliability:

99% → baseline  
 99.9% → 10x cost  
 99.99% → 100x cost  
 99.999% → 1000x cost

The SRE approach: determine the **minimum reliability** your users need to be successful, target that, and spend the saved engineering time on features and innovation.

### 1.2.2 Error Budgets

The error budget is the innovation engine of SRE. It resolves the fundamental tension between:

- **Development:** wants to ship changes fast
- **Operations:** wants to keep the system stable

**Formula:**

Error Budget = 100% – SLO Target

The error budget is the amount of unreliability the team has agreed to tolerate. While budget remains, changes are safe. When the budget is exhausted, all non-critical changes stop until the next window.

The error budget transforms a philosophical debate (how much risk is acceptable?) into a numerical decision (we have 43 minutes of budget left this month).

### 1.2.3 Toil

**Definition:** Toil is operational work that is: - **Manual** — performed by a human - **Repetitive** — done frequently - **Automatable** — a machine could do it - **Tactical** — fixates on individual tasks rather than systemic

improvement - **No enduring value** — the service is no better after the work is done

**The 50% Rule:** At Google, SRE teams enforce that no more than 50% of engineering time is spent on toil. Above 50%, a team cannot make meaningful progress on reliability improvements.

---

## 1.3 The SRE Maturity Model

Understanding where your organisation is on the SRE maturity curve:

| Stage | Name         | Characteristics                                                                         |
|-------|--------------|-----------------------------------------------------------------------------------------|
| 0     | Reactive     | Firefighting mode. No SLOs. Incidents handled by whoever notices. No monitoring.        |
| 1     | Proactive    | Basic monitoring. Incident response process exists. Some automation.                    |
| 2     | Automated    | Error budgets defined. Toil measured and reduced. Automated deployments.                |
| 3     | Data-driven  | SLO-based alerting (burn rates). Predictive monitoring. Chaos engineering.              |
| 4     | Self-healing | Systems auto-remediate. Incidents are rare and brief. SRE is embedded in product teams. |

**Assessment:** Where is Union Bank today? Where should each service be in 12 months?

---

## 1.4 Error Budgets in Depth

### 1.4.1 Calculation

For an availability-based SLO over a time window:

Error Budget (minutes) = (100% – SLO%) × Total Minutes in Window

**Reference table:**

| SLO     | Monthly Budget | Weekly Budget | Annual Budget | Daily Budget |
|---------|----------------|---------------|---------------|--------------|
| 99%     | 7h 18m         | 1h 41m        | 3d 15h        | 14m 24s      |
| 99.5%   | 3h 39m         | 50m           | 1d 19h        | 7m 12s       |
| 99.9%   | 43m 12s        | 10m 4s        | 8h 45m 36s    | 1m 26s       |
| 99.95%  | 21m 36s        | 5m 2s         | 4h 22m 48s    | 43s          |
| 99.99%  | 4m 19s         | 1m 0s         | 52m 34s       | 8.6s         |
| 99.999% | 26s            | 6s            | 5m 15s        | 0.86s        |

### 1.4.2 Burn Rate

Burn rate is how fast the error budget is being consumed relative to the SLO window:

Burn Rate = Budget Consumed / Time Elapsed × (SLO Window / Total Budget)

- **Burn Rate = 1.0** → exactly on track to exhaust budget at end of window

- **Burn Rate** > 1.0 → will exhaust budget before window ends
- **Burn Rate** < 1.0 → on track to have budget remaining

### 1.4.3 Multi-Window Burn Rate Alerting

The correct way to alert on error budget is not a static threshold — it's multi-window, multi-burn-rate alerting (Google's approach):

| Burn Rate | Window                | Action                            |
|-----------|-----------------------|-----------------------------------|
| > 2       | 1 hour (short)        | Page — critical, fast-burning     |
| > 1       | 6 hours (long)        | Page — budget will exhaust        |
| > 0.5     | 3 days (very long)    | Ticket — investigate degradation  |
| > 0.1     | 30 days (full window) | Alert — slow-burn, emerging trend |

#### Example Prometheus alerting rules:

```
# Burn rate > 2 over 1 hour (fast burn, critical)
- alert: ErrorBudgetFastBurn
  expr: |
    (
      1 - (sum(rate(requests_total{status!~"2.."}[1h]))
        / sum(rate(requests_total[1h])))
    )
    /
    (1 - 0.999) # SLO = 99.9%
  > 2
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Error budget burn rate is > 2x over 1 hour"

# Burn rate > 1 over 6 hours (will exhaust budget)
- alert: ErrorBudgetExhaustion
  expr: |
    (
      1 - (sum(rate(requests_total{status!~"2.."}[6h]))
        / sum(rate(requests_total[6h])))
    )
    /
    (1 - 0.999)
  > 1
  for: 5m
  labels:
    severity: warning
```

## 1.5 SRE in the Banking Context

Banks are uniquely positioned to benefit from SRE because:

#### Banking Reality

Regulatory reporting of outages

#### SRE Solution

Quantified SLOs + error budget tracking = compliance data

**Banking Reality**

Financial transactions are irreversible  
 Legacy systems make change risky  
 Customer trust is the product  
 End-of-month salary spikes  
 CBN audit requirements

**SRE Solution**

Correctness SLIs + chaos testing of recovery paths  
 Error budgets gate changes by risk level  
 Blameless culture preserves team trust during incidents  
 Capacity planning driven by SLI trends  
 Infrastructure as Code = auditable change history

**Case Study: The 43-Minute Outage**

A Nigerian bank experienced a 43-minute core banking outage on a Friday afternoon. The root cause: a database connection pool exhausted by an unannounced marketing campaign.

**Traditional response:** Blame the dev who deployed. Firefight. Hope it doesn't happen again.

**SRE response:** - Measure: 43 minutes consumed from 99.9% SLO budget (monthly budget fully exhausted) - Policy: Feature freeze until next month - Action: Add connection pool alerting, capacity review process, marketing-SRE communication channel - Outcome: Systematically prevented recurrence. Marketing now coordinates with SRE before campaigns.

**1.6 Classroom Activity: SRE Maturity Assessment**

**Time:** 45 minutes

**Part A (20 min) — Individual Assessment:**

Assess your current team against the SRE maturity model (Stages 0–4) across these dimensions: 1. Incident response process 2. Monitoring and alerting 3. Automation coverage 4. Error budget usage (if any) 5. Blameless culture 6. Capacity planning

**Part B (25 min) — Group Discussion:**

In groups of 3-4, compare assessments and identify: 1. What stage is Union Bank at today? 2. What is the single highest-impact change to move to the next stage? 3. What would that change cost? What would it save?

**Key Terms**

| Term                     | Definition                                                      |
|--------------------------|-----------------------------------------------------------------|
| <b>SRE</b>               | Applying software engineering to operations problems            |
| <b>Error Budget</b>      | 100% — SLO%; the allowed unreliability in a given window        |
| <b>Burn Rate</b>         | Rate of error budget consumption relative to the SLO window     |
| <b>Toil</b>              | Manual, repetitive, automatable work with no lasting value      |
| <b>Blameless Culture</b> | Investigating systems and processes, not people, after failures |
| <b>SRE Maturity</b>      | Organisational capability level from reactive to self-healing   |
| <b>50% Rule</b>          | Teams should spend no more than half their time on toil         |

**Further Reading**

- [Google SRE Book, Chapter 1 — Introduction](#)
- [Google SRE Workbook, Chapter 5 — Alerting on SLOs](#)
- [SRE Maturity Model — Dynatrace](#)
- [Burn Rate Alerting — Google CRE Life](#)